

03 · Web Server

이 장에서는 순수 **http** 모듈로 서버와 라우터를 직접 만듭니다. 다음 장에서 Express를 쓸 때 "아, 이걸 대신해 주는구나"를 느끼는 것이 목적입니다.

학습 목표

- HTTP 상태 코드를 분류별로 설명하고 상황에 맞게 고른다
- HTTP 메서드와 멱등성을 이해한다
- **http.createServer**로 서버를 띄우고 요청을 처리한다
- 쿼리스트링을 파싱하고 경로별로 분기한다
- 라우팅 테이블 패턴을 직접 구현한다

HTTP 상태 코드

코드	분류	대표 예시
2xx	성공	200 OK · 201 Created · 204 No Content
3xx	리다이렉션	301 영구 이동 · 302 임시 이동 · 304 캐시 사용
4xx	클라이언트 잘못	400 Bad Request · 401 인증 필요 · 403 권한 없음 · 404 없음
5xx	서버 잘못	500 Internal Server Error · 502 Bad Gateway · 503 서비스 불가

401과 **403**을 자주 헷갈립니다. **401**은 "너 누구야?"(인증이 안 됨), **403**은 "너인 건 알겠는데 권한이 없어"(인가 실패)입니다.

HTTP 메서드

메서드	역할	멱등성	본문
GET	리소스 조회	O	없음
POST	리소스 생성	X	있음
PUT	리소스 전체 수정	O	있음
PATCH	리소스 부분 수정	X	있음
DELETE	리소스 삭제	O	없음

1. 가장 단순한 서버

http.createServer(콜백)이 요청마다 (**req**, **res**)를 넘겨줍니다. **req**는 클라이언트가 보낸 것, **res**는 우리가 보낼 것입니다.

03-http/01-simple-server.js

```

/**
 * 가장 단순한 HTTP 서버
 *
 * http.createServer(콜백) 이 요청마다 (req, res) 를 넘겨준다.
 * req: 클라이언트가 보낸 요청 (url, method, headers ...)
 * res: 우리가 보낼 응답. 반드시 res.end() 로 끝내야 한다.
 *
 * 실행: node 03-http/01-simple-server.js
 * 확인: http://localhost:3000
 */
const http = require('http');

const server = http.createServer((req, res) => {
  console.log(`${req.method} ${req.url}`);

  // 상태 코드 + 헤더. 한글이 깨지지 않게 charset=utf-8 을 꼭 붙인다.
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.write('<h1>안녕하세요</h1>');
  res.end('<h1>Hello Node!</h1>'); // end 로 응답 종료
});

server.listen(3000, () => {
  console.log('Listening Server at http://localhost:3000');
});

/*
 자주 쓰는 HTTP 상태 코드
 2xx 성공          200 OK, 201 Created, 204 No Content
 3xx 리다이렉션  301 영구 이동, 302 임시 이동, 304 Not Modified(캐시)
 4xx 클라이언트   400 Bad Request, 401 인증 필요, 403 권한 없음, 404 Not Found
 5xx 서버         500 Internal Server Error, 502 Bad Gateway, 503 서비스 불가

HTTP 메서드
GET      조회
POST     생성
PUT      전체 수정
PATCH   부분 수정
DELETE   삭제
*/

```

실행

```

node 03-http/01-simple-server.js
# 브라우저로 http://localhost:3000 접속

```

참고 할 것

- **res.end()**를 반드시 불러야 합니다. 안 부르면 브라우저가 계속 로딩합니다
- **charset=utf-8**을 빠뜨리면 한글이 ???로 깨집니다. Express의 **res.send**는 이것 자동으로 붙여 줍니다

- `res.write`는 여러 번, `res.end`는 마지막 한 번

2. HTML 파일 서빙과 JSON 응답

경로에 따라 HTML 파일을 읽어 내려주거나 JSON을 돌려줍니다. POST 요청의 본문은 스트림으로 나눠서 도착하므로 조각을 모아야 합니다.

03-http/02-file-server.js

```
/**
 * HTML 파일 서빙 + 경로별 JSON 응답
 *
 * 실행: node 03-http/02-file-server.js
 * 확인: http://localhost:3000      -> index.html
 *       http://localhost:3000/person -> JSON
 */
const http = require('http');
const fs = require('fs').promises;
const path = require('path');
const url = require('url');

http
  .createServer(async (req, res) => {
    const pathname = url.parse(req.url).pathname;

    try {
      if (req.method === 'GET') {
        if (pathname === '/') {
          // 파일을 읽어 그대로 응답 본문으로 내보낸다.
          const data = await fs.readFile(path.join(__dirname, 'index.html'));
          res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
          return res.end(data);
        }

        if (pathname === '/person') {
          res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
          // 객체는 반드시 JSON 문자열로 바꿔서 보낸다.
          return res.end(
            JSON.stringify({ name: '홍길동', age: 30, email: 'a@naver.com' })
          );
        }
      }

      if (req.method === 'POST') {
        // 요청 본문은 스트림으로 나눠서 도착하므로 모아서 처리한다.
        let body = '';
        req.on('data', (chunk) => (body += chunk));
        req.on('end', () => {
```

```

        res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
        res.end(JSON.stringify({ received: body }));
    });
    return;
}

res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
res.end('404 Not Found');
} catch (err) {
    res.writeHead(500, { 'Content-Type': 'text/plain; charset=utf-8' });
    res.end('500 Internal Server Error: ' + err.message);
}
})
.listen(3000, () => {
    console.log('server running at http://localhost:3000');
});

```

실행

```
node 03-http/02-file-server.js
```

다른 터미널에서

```
curl http://localhost:3000/person
```

```
curl -X POST -d "hello" http://localhost:3000/
```

짚고 갈 것

- 객체는 `JSON.stringify()`로 문자열로 바꿔 보냅니다
- 요청 본문은 `req.on("data")`로 모아 `req.on("end")`에서 처리합니다 — Express의 `express.json()`이 이 일을 대신합니다
- 파일 경로는 `path.join(__dirname, ...)` 기준으로. 상대경로는 실행 위치를 따릅니다
- 404와 에러 처리를 빠뜨리면 없는 경로 요청 시 응답 없이 멈춥니다

3. 라우터 직접 만들기

`if/else`로 경로를 분기하면 경로가 늘어날수록 길어지고 중복이 생깁니다. 경로를 **key**, 핸들러를 **value**로 갖는 객체 (**urlMap**)로 바꾸면 한 줄씩만 늘어납니다.

Express의 라우팅도 결국 이 구조를 훨씬 정교하게 만든 것입니다.

03-http/03-router.js

```

/**
 * 순수 http 모듈로 라우터 만들기
 *
 * if/else 로 분기하면 경로가 늘어날수록 길어진다.
 * -> 경로를 key 로, 핸들러를 value 로 갖는 맵(urlMap)으로 정리한다.
 * Express 의 라우팅이 하는 일이 결국 이것이다.

```

```
*
* 실행: node 03-http/03-router.js
* 확인: http://localhost:3000/
*       http://localhost:3000/user?name=홍길동&age=30
*       http://localhost:3000/feed
*/
const http = require('http');
const url = require('url');

// ----- 핸들러 -----
const home = (req, res) => res.end('<h1>HOME</h1>');

const user = (req, res) => {
  // 쿼리스트링(?name=...&age=...)을 객체로 파싱
  const query = url.parse(req.url, true).query;
  res.end(`[user] name: ${query.name}, age: ${query.age}`);
};

const feed = (req, res) => {
  res.end(`<ul>
    <li>picture1</li>
    <li>picture2</li>
    <li>picture3</li>
  </ul>`);
};

const notFound = (req, res) => {
  res.statusCode = 404;
  res.end('404 page not found');
};

// ----- 라우팅 테이블 -----
const urlMap = {
  '/': home,
  '/user': user,
  '/feed': feed,
};

http
  .createServer((req, res) => {
    const path = url.parse(req.url, true).pathname;
    res.setHeader('Content-Type', 'text/html; charset=utf-8');

    if (path in urlMap) {
      urlMap[path](req, res);
    } else {
      notFound(req, res);
    }
  })
  .listen(3000, () => console.log('라우터 서버 시작: http://localhost:3000'));
```

실행

```
node 03-http/03-router.js

curl "http://localhost:3000/user?name=wizard&age=30"
curl http://localhost:3000/feed
curl -i http://localhost:3000/none    # 404
```

짚고 갈 것

- `url.parse(req.url, true)`의 두 번째 인자 `true`가 쿼리를 객체로 파싱합니다
- `url.parse`는 deprecated입니다. 새 코드는 `new URL(req.url, `http://${req.headers.host}`)`
- `path in urlMap`으로 등록 여부를 확인하고, 없으면 404 핸들러로 보냅니다

Express가 대신해 주는 일

다음 장으로 넘어가기 전에, 방금 직접 한 일들을 정리해 둡니다.

직접 한 일	Express에서는
<code>res.writeHead(200, { "Content-Type": ... })</code>	<code>res.json(obj)</code> 한 줄
<code>JSON.stringify(obj)</code>	<code>res.json(obj)</code> 이 자동 처리
<code>req.on("data")</code> 로 본문 모으기	<code>app.use(express.json())</code>
<code>url.parse(req.url, true).query</code>	<code>req.query</code>
<code>urlMap</code> 객체로 분기	<code>app.get(path, handler)</code>
경로에서 값 꺼내기 (직접 파싱)	<code>req.params + /:id</code>
404 분기 직접 작성	마지막 <code>app.use(...)</code>